UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Mini Project Report

On

API Health Log Analyzer Using Python

SUBMITTED BY:

[Your Name]

UID: [Your UID]

CLASS: MCA (CC&D)

SECTION: [Your Section]

SUBMITTED TO:

[Faculty Name]

[Designation]

Table of Content

S. No.	Name	Page no.
1	Acknowledgement	3
2	Introduction and Background	4
3	Objectives	4
4	Technologies Used	4-5
5	Requirements and System Design	5-7
6	Source Code and Output	8-23
7	Functionalities	24
8	Conclusion, Future Work and Learning Outcomes	25
9	GitHub Link and Bibliography	25

Acknowledgement

I would like to express my heartfelt gratitude to all those who contributed to the successful development of the API Health Log Analyzer, a Python tool and website that turn raw API log files into clear health reports.

First and foremost, I am thankful to my mentors and instructors for their continuous guidance and encouragement throughout this project. Their teaching of Python programming and problem solving has been invaluable in shaping the technical direction of this work.

I would also like to acknowledge my peers and friends for their constructive feedback and support during the testing and debugging phases. Their suggestions helped improve the usability and the output of the tool.

Lastly, I am grateful to the open-source community and the developers behind the technologies used in this project. Python and its standard library, along with the Flask framework, made it possible to build this idea with both simplicity and clarity.

This project stands as a testament to curiosity and hands-on learning, and I am proud to present it as a step forward in building practical monitoring tools.

1. Introduction

This report describes the development of the "API Health Log Analyzer", a Python program that reads the log file of a web API, measures how healthy each API endpoint is and presents the result as a console report and as JSON and CSV files. A small Flask website lets anyone upload a log file in the browser and see the result as a dashboard with charts. The project shows how a clear program can replace the slow and error-prone job of reading thousands of log lines by hand.

2. Background

Every time a client calls an API, the server records the request in a log file: when it happened, which endpoint was called, which status code was returned and how long the response took. A busy API can write thousands of such lines in a single hour. Failing requests (status codes 4xx and 5xx) and slow responses are signs of an unhealthy service, but they are easy to miss when the log is read manually.

This project takes inspiration from monitoring tools used in industry and reimagines a simple version of them for learning. The program uses regular expressions to parse the log, dictionaries and the statistics module to calculate the metrics, rule-based classification to label every endpoint and a dashboard page with charts, drawn by Python, to display the final result. The same analysis code powers both the command-line tool and the website, so nothing is written twice.

3. Objectives

- Read an API log file and extract the method, endpoint, status code and response time from every line, skipping malformed lines safely.
- Calculate overall and per-endpoint measures: success rate, error rate, and average, minimum, maximum and 95th percentile response time.
- Classify every endpoint as HEALTHY, DEGRADED or CRITICAL using clear and adjustable rules.
- Present the results as a console report and as JSON and CSV files.
- Provide a small working website where a log file can be uploaded and its health is shown as a dashboard with charts.
- Build a simple, readable and maintainable program that uses first-semester Python concepts.

4. Technologies Used

- **Language:** Python 3 for all the logic of the project.
- **Standard library:** re (regular expressions), statistics, collections (defaultdict, Counter), csv, json, argparse, random, datetime and html.
- **Web framework:** Flask, a lightweight Python framework, for the website routes and file upload.
- **Dashboard:** HTML5 and CSS3 generated by Python, with every chart drawn as inline SVG, so no chart library and no internet connection are needed.

- **Tools:** VS Code or any code editor, and a terminal to run the program.

5. Requirements

- Python 3.8 or higher installed on the computer.
- A code editor (for example VS Code) and a terminal for running the scripts.
- A modern web browser (for example Chrome or Firefox) to view the dashboard and the website.
- Flask (`pip install flask`) for the website. The command-line analyzer needs no installation.
- An API log file in the format described in the System Design section, or the sample file created by `generate_logs.py`.

6. System Design

Architecture Overview

The API Health Log Analyzer follows a simple input, process and output architecture. The input is a plain-text log file, either read from disk or uploaded through the website. The processing layer parses the text, groups the entries by endpoint, calculates the metrics and applies the health rules. The output layer then presents the results as a console report, files or a dashboard web page. Every stage is a separate Python function, so each part can be tested and explained on its own.

Component Overview

- **Input layer:** `api_health.log`, with one request per line, created by `generate_logs.py` or taken from a real server, or a file uploaded in the browser.
- **Processing layer:**
  - `parse_lines()` matches every line with a regular expression and counts malformed lines.
  - `analyze()` and `overall_summary()` calculate the metrics.
  - `health_status()` applies the rules and returns the status.
- **Output layer:** console report, `report.json` and `report.csv`, and the dashboard page of the website.
- **Web layer:** `app.py` (Flask) receives the uploaded file and calls the same processing functions. `dashboard.py` then builds the dashboard page with its SVG charts.

Website Routes

Route	Method	Purpose
/	GET	Shows the upload page
/analyze	POST	Analyzes the uploaded file and shows the dashboard
/sample	GET	Analyzes the sample log (api_health.log)
/api/analyze	POST	Takes log text in the request body and returns the report as JSON

Data Design

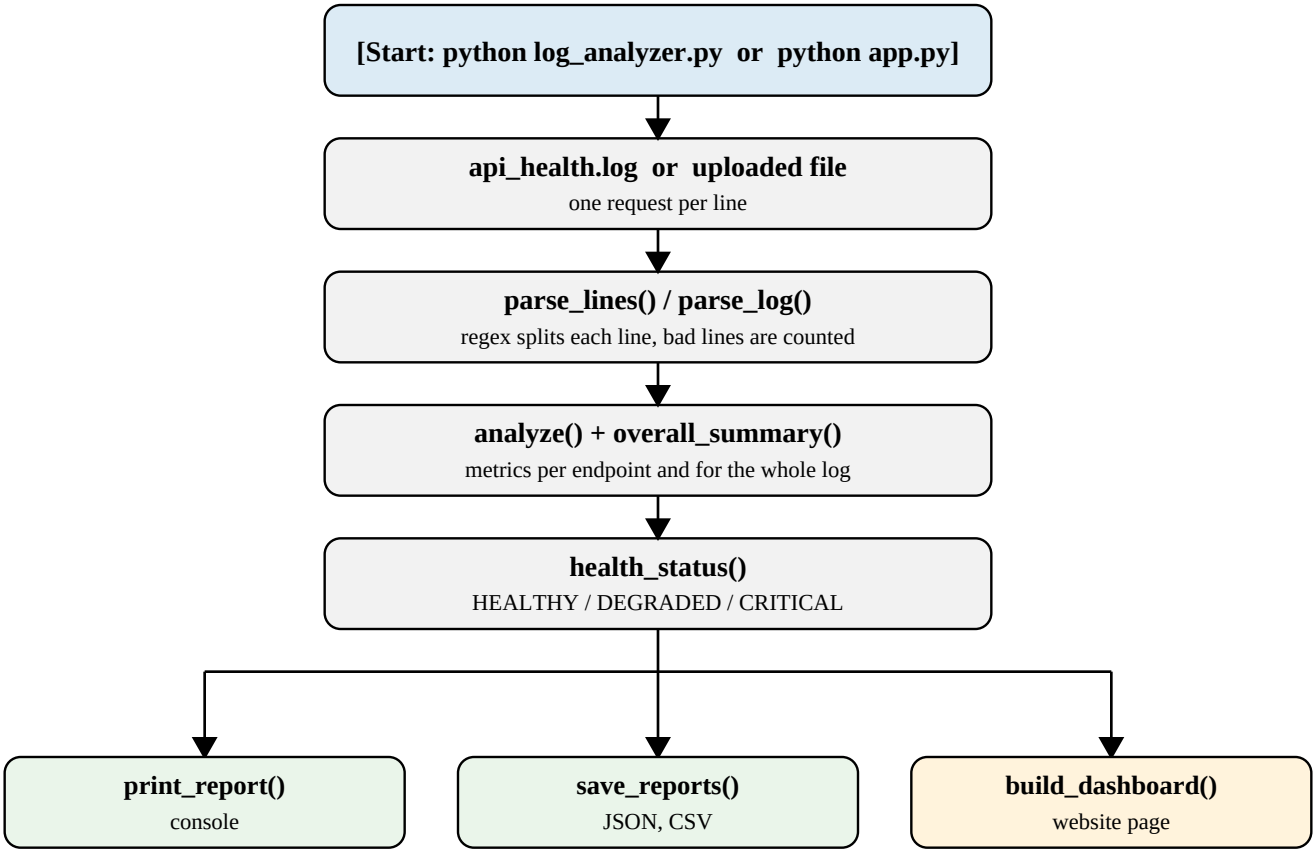
The project uses files instead of a database. Each log line has five fields:

Field	Example	Description
Date and time	2026-10-03 08:00:12	When the request was received
Method	GET	HTTP method of the request
Endpoint	/api/users	The API route that was called
Status code	200	Result returned by the server
Response time	120	Time taken to respond, in milliseconds

The health classification uses these rules, which are defined in `health_status()` and can be changed easily:

Status	Condition
CRITICAL	Error rate above 15% or average response time above 1500 ms
DEGRADED	Error rate above 5% or average response time above 600 ms
HEALTHY	Everything else

Data Flow


[Start: python log_analyzer.py or python app.py]
api_health.log or uploaded file
one request per line
parse_lines() / parse_log()
regex splits each line, bad lines are counted
analyze() + overall_summary()
metrics per endpoint and for the whole log
health_status()
HEALTHY / DEGRADED / CRITICAL
print_report()
console
save_reports()
JSON, CSV
build_dashboard()
website page

7. Source Code & Output

Log generator code (generate_logs.py):-

```
"""
generate_logs.py
Creates a realistic sample API log file (api_health.log) so the analyzer
can be demonstrated without a real server.

Log line format:
    YYYY-MM-DD HH:MM:SS | METHOD | /endpoint | STATUS | RESPONSE_TIME_MS
"""

import random
from datetime import datetime, timedelta

# endpoint -> (failure chance, normal response time range in ms)
ENDPOINTS = {
    "/api/login":    (0.05, (80, 250)),
    "/api/users":    (0.02, (60, 200)),
    "/api/products": (0.03, (100, 400)),
    "/api/orders":   (0.12, (200, 900)),   # intentionally unhealthy
    "/api/payments": (0.25, (400, 2500)),  # intentionally critical
    "/api/search":   (0.04, (150, 600)),
}
METHODS = ["GET", "POST", "PUT", "DELETE"]
OK_CODES = [200, 200, 200, 201, 204]
CLIENT_ERRORS = [400, 401, 403, 404]
SERVER_ERRORS = [500, 502, 503, 504]


def generate(filename="api_health.log", total=1000, seed=42):
    random.seed(seed)
    current = datetime(2026, 10, 3, 8, 0, 0)

    with open(filename, "w") as f:
        for _ in range(total):
            current += timedelta(seconds=random.randint(1, 30))
            endpoint = random.choice(list(ENDPOINTS))
            fail_chance, (low, high) = ENDPOINTS[endpoint]

            if random.random() < fail_chance:
                # failed request: 40% client error, 60% server error
                if random.random() < 0.4:
                    status = random.choice(CLIENT_ERRORS)
                else:
                    status = random.choice(SERVER_ERRORS)
                response_time = random.randint(high, high * 2)   # slow on failure
            else:
                status = random.choice(OK_CODES)
                response_time = random.randint(low, high)

            f.write(
                f"{current:%Y-%m-%d %H:%M:%S} | {random.choice(METHODS)} | "
                f"{endpoint} | {status} | {response_time}\n"
            )
    print(f"Generated {total} log entries in '{filename}'")


if __name__ == "__main__":
    generate()
```

Main program code (log_analyzer.py):-

```
"""
log_analyzer.py  -  API Health Log Analyzer
-------------------------------------------
Reads an API log file, calculates health metrics (success rate, error rate,
response times) for every endpoint, assigns a health status, and saves
the report as text, CSV and JSON.

Usage:
    python log_analyzer.py                      # analyzes api_health.log
    python log_analyzer.py --file mylog.log
    python log_analyzer.py --slow 800 --top 3
"""

import argparse
import csv
import json
import re
import statistics
from collections import Counter, defaultdict

# ---------- Configuration ----------
LOG_PATTERN = re.compile(
    r"^(?P<date>\d{4}-\d{2}-\d{2}) (?P<time>\d{2}:\d{2}:\d{2}) \| "
    r"(?P<method>[A-Z]+) \| (?P<endpoint>\S+) \| "
    r"(?P<status>\d{3}) \| (?P<rt>\d+)$"
)


# ---------- Step 1: Parse ----------
def parse_lines(lines):
    """Parse an iterable of log lines. Returns (valid entries, number of bad lines)."""
    entries, bad_lines = [], 0
    for line in lines:
        line = line.strip()
        if not line:
            continue
        match = LOG_PATTERN.match(line)
        if match:
            entries.append({
                "hour": match["time"][:2],
                "method": match["method"],
                "endpoint": match["endpoint"],
                "status": int(match["status"]),
                "rt": int(match["rt"]),
            })
        else:
            bad_lines += 1
    return entries, bad_lines


def parse_log(filename):
    """Read the file and return (list of valid entries, number of bad lines)."""
    try:
        with open(filename) as f:
            return parse_lines(f)
    except FileNotFoundError:
        print(f"Error: file '{filename}' not found. Run generate_logs.py first.")
        raise SystemExit(1)
```

```
# ---------- Step 2: Calculate metrics ----------
def percentile(values, p):
    """Simple percentile (e.g. p=95) without external libraries."""
    ordered = sorted(values)
    index = max(0, int(round(p / 100 * len(ordered))) - 1)
    return ordered[index]


def health_status(error_rate, avg_rt):
    """Rule-based health classification."""
    if error_rate > 15 or avg_rt > 1500:
        return "CRITICAL"
    if error_rate > 5 or avg_rt > 600:
        return "DEGRADED"
    return "HEALTHY"


def analyze(entries):
    by_endpoint = defaultdict(list)
    for e in entries:
        by_endpoint[e["endpoint"]].append(e)

    results = []
    for endpoint, rows in by_endpoint.items():
        total = len(rows)
        times = [r["rt"] for r in rows]
        client_err = sum(1 for r in rows if 400 <= r["status"] < 500)
        server_err = sum(1 for r in rows if r["status"] >= 500)
        error_rate = (client_err + server_err) / total * 100
        avg_rt = statistics.mean(times)
        results.append({
            "endpoint": endpoint,
            "requests": total,
            "success_rate": round(100 - error_rate, 2),
            "error_rate": round(error_rate, 2),
            "4xx": client_err,
            "5xx": server_err,
            "avg_ms": round(avg_rt, 1),
            "min_ms": min(times),
            "max_ms": max(times),
            "p95_ms": percentile(times, 95),
            "status": health_status(error_rate, avg_rt),
        })
    return sorted(results, key=lambda r: r["error_rate"], reverse=True)


def overall_summary(entries):
    total = len(entries)
    errors = sum(1 for e in entries if e["status"] >= 400)
    times = [e["rt"] for e in entries]
    return {
        "total_requests": total,
        "total_errors": errors,
        "overall_error_rate": round(errors / total * 100, 2),
        "overall_success_rate": round(100 - errors / total * 100, 2),
        "avg_response_ms": round(statistics.mean(times), 1),
        "median_response_ms": statistics.median(times),
        "p95_response_ms": percentile(times, 95),
        "status_codes": dict(sorted(Counter(e["status"] for e in entries).items())),
        "errors_by_hour": dict(sorted(Counter(
```

```
            e["hour"] for e in entries if e["status"] >= 400).items())),
    }


# ---------- Step 3: Report ----------
def print_report(summary, results, bad_lines, slow_limit, top_n, entries):
    line = "=" * 78
    print(line)
    print("                     API HEALTH LOG ANALYSIS REPORT")
    print(line)
    print(f"Total requests     : {summary['total_requests']}")
    print(f"Success rate       : {summary['overall_success_rate']}%")
    print(f"Error rate         : {summary['overall_error_rate']}%")
    print(f"Avg response time  : {summary['avg_response_ms']} ms "
          f"(median {summary['median_response_ms']}, p95 {summary['p95_response_ms']})")
    print(f"Malformed lines    : {bad_lines}")

    print("\nStatus code distribution:")
    for code, count in summary["status_codes"].items():
        bar = "#" * int(count / summary["total_requests"] * 100)
        print(f"  {code}  {count:>5}  {bar}")

    print("\nPer-endpoint health:")
    print(f"  {'Endpoint':<16}{'Reqs':>6}{'Err%':>8}{'Avg ms':>9}{'P95 ms':>9}  Status")
    print("  " + "-" * 62)
    for r in results:
        print(f"  {r['endpoint']:<16}{r['requests']:>6}{r['error_rate']:>8}"
              f"{r['avg_ms']:>9}{r['p95_ms']:>9}  {r['status']}")

    print(f"\nTop {top_n} most error-prone endpoints:")
    for i, r in enumerate(results[:top_n], 1):
        print(f"  {i}. {r['endpoint']}  ({r['error_rate']}% errors)")

    slow = [e for e in entries if e["rt"] > slow_limit]
    print(f"\nSlow requests (> {slow_limit} ms): {len(slow)}")

    print("\nErrors by hour:")
    for hour, count in summary["errors_by_hour"].items():
        print(f"  {hour}:00  {count}")

    worst = [r for r in results if r["status"] != "HEALTHY"]
    print("\nALERTS:")
    if worst:
        for r in worst:
            print(f"  [{r['status']}] {r['endpoint']} - "
                  f"{r['error_rate']}% errors, avg {r['avg_ms']} ms")
    else:
        print("  All endpoints are healthy.")
    print(line)


def save_reports(summary, results):
    with open("report.json", "w") as f:
        json.dump({"summary": summary, "endpoints": results}, f, indent=4)
    with open("report.csv", "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=results[0].keys())
        writer.writeheader()
        writer.writerows(results)
    print("Saved: report.json, report.csv")
```

```
# ---------- Main ----------
def main():
    parser = argparse.ArgumentParser(description="API Health Log Analyzer")
    parser.add_argument("--file", default="api_health.log", help="log file path")
    parser.add_argument("--slow", type=int, default=1000, help="slow request limit (ms)")
    parser.add_argument("--top", type=int, default=3, help="how many top error endpoints")
    args = parser.parse_args()

    entries, bad_lines = parse_log(args.file)
    if not entries:
        print("No valid log entries found.")
        return

    results = analyze(entries)
    summary = overall_summary(entries)
    print_report(summary, results, bad_lines, args.slow, args.top, entries)
    save_reports(summary, results)


if __name__ == "__main__":
    main()
```

Dashboard code (dashboard.py):-

```
"""
dashboard.py  -  builds the dashboard page shown by the website
------------------------------------------------------------
Every chart is drawn as inline SVG by plain Python, so the page needs no
chart library and no internet connection. app.py calls build_dashboard().
"""

from html import escape

COLORS = {"HEALTHY": "#2E9E6B", "DEGRADED": "#F2A541", "CRITICAL": "#D64545"}
TINTS = {"HEALTHY": "#DDF3E8", "DEGRADED": "#FDF0D9", "CRITICAL": "#FBE3E3"}
INKS = {"HEALTHY": "#1F7A4D", "DEGRADED": "#8A5200", "CRITICAL": "#B02A2A"}

ICONS = {
    "activity": '<polyline points="22 12 18 12 15 21 9 3 6 12 2 12"/>',
    "check": '<path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"/><polyline points="22 4 12 14.01 9 11.01"/>',
    "alert": ('<path d="M10.29 3.86 1.82 18a2 2 0 0 0 1.71 3h16.94a2 2 0 0 0 1.71-3L13.71 3.86a2 2 0 0 0-3.42 0z"/>'
              '<line x1="12" y1="9" x2="12" y2="13"/><line x1="12" y1="17" x2="12.01" y2="17"/>'),
    "clock": '<circle cx="12" cy="12" r="10"/><polyline points="12 6 12 12 16 14"/>',
    "zap": '<polygon points="13 2 3 14 12 14 11 22 21 10 12 10 13 2"/>',
    "upload": ('<polyline points="16 16 12 12 8 16"/><line x1="12" y1="12" x2="12" y2="21"/>'
               '<path d="M20.39 18.39A5 5 0 0 0 18 9h-1.26A8 8 0 1 0 3 16.3"/>'),
}
TONES = {"teal": ("#E0F5F7", "#0E9AA7"), "green": ("#DDF3E8", "#1F7A4D"), "red": ("#FBE3E3", "#B02A2A"),
         "amber": ("#FDF0D9", "#B26B00"), "indigo": ("#E6E9FB", "#4650C8")}

CSS = """
:root{--ink:#0B1B2B;--navy:#12304A;--teal:#0E9AA7;--mute:#5B7C99;--line:#E3EAF0}
*{box-sizing:border-box}
body{margin:0;font-family:"Segoe UI",system-ui,Arial,sans-serif;color:var(--ink);min-height:100vh;
background:radial-gradient(circle at 10% -10%,#D7F0F2 0,transparent 38%),
radial-gradient(circle at 95% 0,#E3E9FA 0,transparent 35%),#F2F6FA}
svg.i{fill:none;stroke:currentColor;stroke-width:2;stroke-linecap:round;stroke-linejoin:round}
.top{background:linear-gradient(120deg,#0B1B2B 0%,#12304A 55%,#0E7C89 100%);color:#fff;padding:20px 32px;
display:flex;flex-wrap:wrap;gap:14px;justify-content:space-between;align-items:center}
.brand{display:flex;align-items:center;gap:14px}
.logo{width:48px;height:48px;border-radius:14px;background:linear-gradient(135deg,#19C2CF,#0E9AA7);color:#fff;
display:grid;place-items:center;box-shadow:0 4px 16px rgba(14,154,167,.5)}.logo svg{width:26px;height:26px}
.top h1{margin:0;font-size:22px;letter-spacing:.2px}
.meta{margin-top:6px;display:flex;flex-wrap:wrap;gap:8px}
.chip{background:rgba(255,255,255,.13);border-radius:20px;padding:3px 12px;font-size:12.5px;color:#D3E8F6}
.btn{background:#fff;color:#0E7C89;border:0;border-radius:10px;padding:10px 20px;font-size:14px;
font-weight:700;text-decoration:none;cursor:pointer;display:inline-block;transition:.15s}
.btn:hover{transform:translateY(-1px);box-shadow:0 6px 16px rgba(0,0,0,.22)}
.wrap{max-width:1200px;margin:0 auto;padding:24px 20px 6px}
.banner{border-radius:16px;padding:18px 22px;margin-bottom:18px;display:flex;align-items:center;gap:16px;
border:1px solid rgba(0,0,0,.05)}.banner b{font-size:19px}.banner .sub{font-size:14px;opacity:.85}
.pulse{width:16px;height:16px;border-radius:50%;flex:none;box-shadow:0 0 0 6px rgba(255,255,255,.7)}
.kpis{display:grid;grid-template-columns:repeat(auto-fit,minmax(190px,1fr));gap:16px;margin-bottom:18px}
.kpi,.card{background:#fff;border-radius:16px;border:1px solid #EDF2F7;transition:.15s;
box-shadow:0 2px 6px rgba(18,48,74,.06),0 8px 24px rgba(18,48,74,.07)}
.kpi:hover,.card:hover{transform:translateY(-2px)}
.kpi{padding:16px 18px;display:flex;align-items:center;gap:14px}
.ico{width:46px;height:46px;border-radius:14px;display:grid;place-items:center;flex:none}.ico svg{width:23px;height:23px}
.kpi b{display:block;font-size:26px;line-height:1.1}.kpi span{color:var(--mute);font-size:13px}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(340px,1fr));gap:18px;margin-bottom:18px}
.card{padding:18px 20px}.card h2{font-size:15.5px;margin:0 0 14px;color:var(--navy)}
.chart{width:100%;height:auto;display:block}.chart .lbl{font-size:11px;fill:#0B1B2B}
.chart .val,.chart .axis{font-size:10px;fill:#5B7C99}.chart .grid{stroke:#E3EAF0;stroke-width:1}
.chart .limit{stroke:#8FB8DE;stroke-width:1;stroke-dasharray:3 3}
```

```
.donutbox{display:flex;align-items:center;gap:18px;flex-wrap:wrap}.donut{width:150px;flex:none}
.donut .big{font-size:6px;font-weight:700;fill:#0B1B2B}.donut .small{font-size:2.6px;fill:#5B7C99}
.legend{list-style:none;margin:0;padding:0;font-size:14px}.legend li{margin:6px 0}
.legend i,.key i{display:inline-block;width:11px;height:11px;border-radius:3px;margin-right:7px}
.key{font-size:13px;color:#5B7C99;margin-bottom:6px}.key i{margin-left:10px}.key i:first-child{margin-left:0}
.alert{margin:10px 0;font-size:14px}.detail{color:#5B7C99;font-size:12px;margin-top:3px}
.empty{color:#5B7C99;font-size:14px}
.scroll{overflow-x:auto}
table{width:100%;border-collapse:collapse;font-size:14px}
th,td{padding:10px 8px;text-align:right;border-bottom:1px solid var(--line)}
th:first-child,td:first-child{text-align:left}
th{color:#5B7C99;font-weight:600;font-size:12.5px;text-transform:uppercase;letter-spacing:.4px}
tbody tr:nth-child(even){background:#F8FBFD}tbody tr:hover{background:#EAF6F8}
.badge{padding:3px 11px;border-radius:12px;font-weight:700;font-size:12px}
.foot{text-align:center;color:#5B7C99;font-size:12.5px;padding:14px 0 26px}
"""


def icon(name):
    return f'<svg class="i" viewBox="0 0 24 24">{ICONS[name]}</svg>'


def nice_top(value):
    """Round a chart maximum up, so the 3 grid steps are round numbers (0, 2000, 4000, 6000)."""
    step = 1
    while True:
        for factor in (1, 2, 5):
            if step * factor * 3 >= value:
                return step * factor * 3
        step *= 10


def donut(parts, big, small):
    """Donut chart. parts is a list of (label, value, color)."""
    total = sum(v for _, v, _ in parts) or 1
    rings, offset = "", 25                       # 25 makes the first slice start at the top
    for label, v, color in parts:
        pct = v / total * 100
        if pct:
            rings += (f'<circle cx="21" cy="21" r="15.9155" fill="none" stroke="{color}" stroke-width="5" '
                      f'stroke-dasharray="{pct:.2f} {100 - pct:.2f}" stroke-dashoffset="{offset:.2f}">'
                      f"<title>{escape(label)}: {v}</title></circle>")
        offset -= pct
    legend = "".join(f'<li><i style="background:{c}"></i>{escape(l)} <b>{v}</b></li>' for l, v, c in parts)
    return ('<div class="donutbox"><svg viewBox="0 0 42 42" class="donut">'
            '<circle cx="21" cy="21" r="15.9155" fill="none" stroke="#EAF2F7" stroke-width="5"/>'
            f'{rings}<text x="21" y="21.8" text-anchor="middle" class="big">{big}</text>'
            f'<text x="21" y="26.5" text-anchor="middle" class="small">{small}</text></svg>'
            f'<ul class="legend">{legend}</ul></div>')


def error_bars(results):
    """Horizontal bars: error rate per endpoint, with the 5% and 15% limits marked."""
    top = max(15, max(r["error_rate"] for r in results)) * 1.1
    rows, row_h = "", 30
    for i, r in enumerate(results):
        y = 8 + i * row_h
        width = r["error_rate"] / top * 215
        rows += (f'<text x="0" y="{y + 12}" class="lbl">{escape(r["endpoint"])}</text>'
                 f'<rect x="100" y="{y}" width="215" height="16" rx="4" fill="#EAF2F7"/>'
```

```
                f'<rect x="100" y="{y}" width="{width:.1f}" height="16" rx="4" fill="{COLORS[r["status"]]}">'
                f'<title>{escape(r["endpoint"])}: {r["error_rate"]}% errors</title></rect>'
                f'<text x="320" y="{y + 12}" class="val">{r["error_rate"]}%</text>')
    height = 8 + len(results) * row_h
    limits = ""
    for limit in (5, 15):
        x = 100 + limit / top * 215
        limits += (f'<line x1="{x:.1f}" y1="0" x2="{x:.1f}" y2="{height}" class="limit"/>'
                   f'<text x="{x:.1f}" y="{height + 11}" text-anchor="middle" class="axis">{limit}%</text>')
    return f'<svg viewBox="0 0 360 {height + 16}" class="chart">{limits}{rows}</svg>'


def time_columns(results):
    """Grouped columns: average and P95 response time per endpoint."""
    top = nice_top(max(r["p95_ms"] for r in results) * 1.05)
    x0, base, chart_h = 40, 150, 130
    grid = ""
    for k in range(4):
        value = top * k / 3
        y = base - value / top * chart_h
        grid += (f'<line x1="{x0}" y1="{y:.1f}" x2="352" y2="{y:.1f}" class="grid"/>'
                 f'<text x="{x0 - 5}" y="{y + 3:.1f}" text-anchor="end" class="axis">{value:.0f}</text>')
    slot = (352 - x0) / len(results)
    bars = ""
    for i, r in enumerate(results):
        cx = x0 + slot * (i + 0.5)
        for dx, key, color in ((-9, "avg_ms", "#0E9AA7"), (1, "p95_ms", "#12304A")):
            h = r[key] / top * chart_h
            bars += (f'<rect x="{cx + dx:.1f}" y="{base - h:.1f}" width="8" height="{h:.1f}" rx="1.5" '
                     f'fill="{color}"><title>{escape(r["endpoint"])} {key}: {r[key]} ms</title></rect>')
        bars += (f'<text x="{cx:.1f}" y="{base + 13}" text-anchor="middle" class="axis">'
                 f'{escape(r["endpoint"].split("/")[-1][:8])}</text>')
    key = ('<div class="key"><i style="background:#0E9AA7"></i>Average'
           '<i style="background:#12304A"></i>P95 (ms)</div>')
    return f'{key}<svg viewBox="0 0 360 165" class="chart">{grid}{bars}</svg>'


def hour_line(hours):
    """Line chart: number of errors in each hour of the log."""
    if not hours:
        return '<div class="empty">No errors recorded.</div>'
    items = list(hours.items())
    top = nice_top(max(n for _, n in items) * 1.1)
    x0, x1, base, chart_h = 32, 345, 135, 110
    points = []
    for i, (hour, n) in enumerate(items):
        x = x0 + (x1 - x0) * (i / (len(items) - 1) if len(items) > 1 else 0.5)
        points.append((x, base - n / top * chart_h, hour, n))
    line = " ".join(f"{x:.1f},{y:.1f}" for x, y, _, _ in points)
    area = f"{points[0][0]:.1f},{base} {line} {points[-1][0]:.1f},{base}"
    grid = ""
    for k in range(4):
        value = top * k / 3
        y = base - value / top * chart_h
        grid += (f'<line x1="{x0}" y1="{y:.1f}" x2="{x1 + 5}" y2="{y:.1f}" class="grid"/>'
                 f'<text x="{x0 - 5}" y="{y + 3:.1f}" text-anchor="end" class="axis">{value:.0f}</text>')
    dots = "".join(f'<circle cx="{x:.1f}" cy="{y:.1f}" r="4" fill="#0E9AA7"><title>{h}:00 - {n} errors</title>'
                   f'</circle><text x="{x:.1f}" y="{base + 14}" text-anchor="middle" class="axis">{h}:00</text>'
                   for x, y, h, n in points)
    return (f'<svg viewBox="0 0 360 155" class="chart">{grid}<polygon points="{area}" fill="#0E9AA7" '
```

```
            f'opacity=".15"/><polyline points="{line}" fill="none" stroke="#0E9AA7" stroke-width="2.5"/>{dots}</svg>')


def kpi(name, tone, value, label):
    bg, fg = TONES[tone]
    return (f'<div class="kpi"><div class="ico" style="background:{bg};color:{fg}">{icon(name)}</div>'
            f"<div><b>{value}</b><span>{label}</span></div></div>")


def badge(status):
    return f'<span class="badge" style="background:{TINTS[status]};color:{INKS[status]}">{status}</span>'


def page_header(title, chips, button=""):
    """The dark gradient bar at the top of every page."""
    chip_html = "".join(f'<span class="chip">{c}</span>' for c in chips)
    return (f'<div class="top"><div class="brand"><div class="logo">{icon("activity")}</div><div>'
            f'<h1>{title}</h1><div class="meta">{chip_html}</div></div></div>{button}</div>')


def build_dashboard(summary, results, name="", bad_lines=0):
    """Return the complete dashboard page as one HTML string."""
    counts = {s: sum(1 for r in results if r["status"] == s) for s in COLORS}
    worst = "CRITICAL" if counts["CRITICAL"] else "DEGRADED" if counts["DEGRADED"] else "HEALTHY"
    banner = (f'<div class="banner" style="background:linear-gradient(90deg,{TINTS[worst]},#fff);'
              f'color:{INKS[worst]}"><span class="pulse" style="background:{COLORS[worst]}"></span><div>'
              f'<b>Overall status: {worst}</b><br><span class="sub">{counts["CRITICAL"]} critical, '
              f'{counts["DEGRADED"]} degraded and {counts["HEALTHY"]} healthy endpoints</span></div></div>')

    codes = summary["status_codes"]
    ok = sum(n for c, n in codes.items() if c < 400)
    client = sum(n for c, n in codes.items() if 400 <= c < 500)
    server = sum(n for c, n in codes.items() if c >= 500)
    health_donut = donut([(s.title(), counts[s], COLORS[s]) for s in COLORS], len(results), "endpoints")
    result_donut = donut([("Success (2xx, 3xx)", ok, "#2E9E6B"), ("Client errors (4xx)", client, "#F2A541"),
                          ("Server errors (5xx)", server, "#D64545")],
                         f'{summary["overall_success_rate"]}%', "success")

    alerts = "".join(
        f'<div class="alert">{badge(r["status"])}  <b>{escape(r["endpoint"])}</b>'
        f'<div class="detail">{r["error_rate"]}% errors, average {r["avg_ms"]} ms</div></div>'
        for r in results if r["status"] != "HEALTHY"
    ) or '<div class="empty">All endpoints are healthy.</div>'

    rows = "".join(
        f'<tr><td>{escape(r["endpoint"])}</td><td>{r["requests"]}</td><td>{r["error_rate"]}%</td>'
        f'<td>{r["4xx"]}</td><td>{r["5xx"]}</td><td>{r["avg_ms"]}</td><td>{r["p95_ms"]}</td>'
        f'<td>{badge(r["status"])}</td></tr>' for r in results)

    kpis = "".join([
        kpi("activity", "teal", summary["total_requests"], "Total requests"),
        kpi("check", "green", f'{summary["overall_success_rate"]}%', "Success rate"),
        kpi("alert", "red", f'{summary["overall_error_rate"]}%', f'Error rate ({summary["total_errors"]} errors)'),
        kpi("clock", "amber", f'{summary["avg_response_ms"]} ms', "Average response"),
        kpi("zap", "indigo", f'{summary["p95_response_ms"]} ms', "95th percentile response"),
    ])
    header = page_header("API Health Dashboard",
                         [escape(name), f'{summary["total_requests"]} requests', f"{bad_lines} skipped lines"],
                         '<a class="btn" href="/">Analyze another file</a>')
```

```
    return (
        '<!DOCTYPE html><html lang="en"><head><meta charset="utf-8">'
        '<meta name="viewport" content="width=device-width, initial-scale=1">'
        f"<title>API Health Dashboard</title><style>{CSS}</style></head><body>{header}"
        f'<div class="wrap">{banner}<div class="kpis">{kpis}</div>'
        '<div class="grid">'
        f'<div class="card"><h2>Endpoint health</h2>{health_donut}</div>'
        f'<div class="card"><h2>Requests by result</h2>{result_donut}</div>'
        f'<div class="card"><h2>Alerts</h2>{alerts}</div></div>'
        '<div class="grid">'
        f'<div class="card"><h2>Error rate by endpoint</h2>{error_bars(results)}</div>'
        f'<div class="card"><h2>Response time by endpoint (ms)</h2>{time_columns(results)}</div>'
        f'<div class="card"><h2>Errors by hour</h2>{hour_line(summary["errors_by_hour"])}</div></div>'
        '<div class="card tablecard"><h2>Endpoint health table</h2><div class="scroll"><table><thead><tr><th>Endpoint</th>'
        '<th>Requests</th><th>Error %</th><th>4xx</th><th>5xx</th><th>Avg ms</th><th>P95 ms</th>'
        f"<th>Status</th></tr></thead><tbody>{rows}</tbody></table></div></div></div>"
        '<div class="foot">API Health Log Analyzer &amp;nbsp;|&amp;nbsp; MCA (CC&amp;amp;D) Python mini project</div>'
        "</body></html>"
    )
```

Website code (app.py):-

```
"""
app.py  -  API Health Log Analyzer (website)
--------------------------------------------
A small Flask website. The visitor uploads an API log file and the site
shows the health dashboard for that file. It reuses the functions of
log_analyzer.py, so no analysis code is repeated.

Run:    pip install flask
        python app.py
Open:   http://127.0.0.1:5000

Routes:
    GET  /              upload page
    POST /analyze       analyze the uploaded file and show the dashboard
    GET  /sample        analyze the sample log (api_health.log)
    POST /api/analyze   send log text in the body, get the report as JSON
"""

from html import escape
from pathlib import Path

from flask import Flask, jsonify, request

import dashboard
import generate_logs
import log_analyzer as la

SAMPLE = Path(__file__).parent / "api_health.log"

app = Flask(__name__)
app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024      # upload limit: 5 MB

EXTRA_CSS = """
.hero{background:linear-gradient(120deg,#0B1B2B 0%,#12304A 55%,#0E7C89 100%);color:#fff;text-align:center;
padding:46px 20px 110px}.hero h1{margin:0 0 10px;font-size:38px;line-height:1.15}
.hero p{margin:0 auto;max-width:560px;color:#BCD8EA;font-size:17px}
.feats{margin-top:20px;display:flex;flex-wrap:wrap;gap:10px;justify-content:center}
.upcard{max-width:640px;margin:-80px auto 0;padding:26px;position:relative}
.dz{display:block;border:2px dashed #8FB8DE;border-radius:14px;padding:30px 20px;text-align:center;
background:#F5FAFD;cursor:pointer;transition:.15s}.dz:hover,.dz.over{background:#E4F4F6;border-color:#0E9AA7}
.dz svg{width:46px;height:46px;color:#0E9AA7}.dz .t{font-size:17px;font-weight:700;margin-top:8px}
.dz .s{color:#5B7C99;font-size:13.5px;margin-top:4px}.dz .f{margin-top:10px;font-weight:700;color:#0E7C89}
.actions{display:flex;flex-wrap:wrap;gap:12px;justify-content:center;align-items:center;margin-top:18px}
button.go{background:linear-gradient(135deg,#19C2CF,#0E9AA7);color:#fff;border:0;border-radius:10px;
padding:12px 28px;font-size:15px;font-weight:700;cursor:pointer;box-shadow:0 6px 16px rgba(14,154,167,.35)}
button.go:hover{transform:translateY(-1px)}
a.alt{color:#0E7C89;border:2px solid #0E9AA7;border-radius:10px;padding:10px 22px;font-weight:700;
text-decoration:none;font-size:15px}a.alt:hover{background:#E4F4F6}
.error{background:#FBE3E3;color:#B02A2A;border-radius:10px;padding:11px 14px;margin-bottom:16px;font-weight:600}
.steps{max-width:900px;margin:34px auto 0;display:grid;grid-template-columns:repeat(auto-fit,minmax(240px,1fr));gap:16px}
.step{padding:18px 20px}.num{width:34px;height:34px;border-radius:50%;background:#0E9AA7;color:#fff;
display:grid;place-items:center;font-weight:700;margin-bottom:10px}.step h2{margin:0 0 6px;font-size:16px}
.step p{margin:0;color:#5B7C99;font-size:14px}
.fmt{max-width:900px;margin:16px auto 0}
code{background:#EAF2F7;padding:3px 8px;border-radius:6px;font-size:13px}
"""

UPLOAD_JS = """
const dz = document.getElementById('dz'), file = document.getElementById('f');
const show = () => { document.getElementById('fname').textContent = file.files.length ? file.files[0].name : ''; };
```

```
file.addEventListener('change', show);
['dragenter', 'dragover'].forEach(e => dz.addEventListener(e, ev => { ev.preventDefault(); dz.classList.add('over'); }));
['dragleave', 'drop'].forEach(e => dz.addEventListener(e, ev => { ev.preventDefault(); dz.classList.remove('over'); }));
dz.addEventListener('drop', ev => { file.files = ev.dataTransfer.files; show(); });
document.getElementById('form').addEventListener('submit', () => {
  document.getElementById('go').textContent = 'Analyzing...';
});
"""


def page(body, title="API Health Log Analyzer", script=""):
    """Wrap some HTML in a complete page that uses the dashboard styles."""
    return ('<!DOCTYPE html><html lang="en"><head><meta charset="utf-8">'
            '<meta name="viewport" content="width=device-width, initial-scale=1">'
            f"<title>{title}</title><style>{dashboard.CSS}{EXTRA_CSS}</style></head>"
            f"<body>{body}<script>{script}</script></body></html>")


def upload_page(error=""):
    error_box = f'<div class="error">{escape(error)}</div>' if error else ""
    chips = "".join(f'<span class="chip">{c}</span>' for c in
                    ("Success and error rates", "Response times", "Charts and alerts", "JSON API"))
    steps = "".join(
        f'<div class="card step"><div class="num">{n}</div><h2>{t}</h2><p>{d}</p></div>'
        for n, t, d in (("1", "Upload", "Choose your API log file, or try the built-in sample."),
                        ("2", "Analyze", "Python parses every line and calculates the health of each endpoint."),
                        ("3", "See the dashboard", "Status banner, charts, alerts and a table appear instantly.")))
    return page(
        dashboard.page_header("API Health Log Analyzer", ["MCA (CC&amp;D) Python mini project"])
        + '<div class="hero"><h1>Check the health of your API<br>in seconds</h1>'
        "<p>Upload an API log file and get a clear dashboard with charts, alerts and response times.</p>"
        f'<div class="feats">{chips}</div></div>'
        f'<div class="card upcard">{error_box}'
        '<form id="form" action="/analyze" method="post" enctype="multipart/form-data">'
        f'<label class="dz" id="dz"><input type="file" name="logfile" id="f" accept=".log,.txt" hidden>'
        f'{dashboard.icon("upload")}<div class="t">Drag and drop your log file here</div>'
        '<div class="s">or click to browse (.log or .txt, up to 5 MB)</div><div class="f" id="fname"></div></label>'
        '<div class="actions"><button class="go" id="go" type="submit">Analyze log</button>'
        '<a class="alt" href="/sample">Try the sample log</a></div></form></div>'
        f'<div class="steps">{steps}</div>'
        '<div class="card fmt"><h2>Expected log format</h2>'
        "<code>2026-10-03 08:00:12 | GET | /api/users | 200 | 120</code>"
        '<p style="color:#5B7C99;font-size:14px;margin-bottom:0">Fields: date, time, method, endpoint, '
        "status code, response time in ms.</p></div>"
        '<div class="foot">API Health Log Analyzer  |  MCA (CC&amp;D) Python mini project</div>',
        script=UPLOAD_JS,
    )


def show_dashboard(text, name):
    """Parse the log text and return the dashboard page (or the upload page with an error)."""
    entries, bad_lines = la.parse_lines(text.splitlines())
    if not entries:
        return upload_page("No valid log lines found. Check the file against the format below."), 400
    results = la.analyze(entries)
    summary = la.overall_summary(entries)
    return dashboard.build_dashboard(summary, results, name, bad_lines)


@app.route("/")
```

```
def home():
    return upload_page()


@app.route("/analyze", methods=["POST"])
def analyze_upload():
    file = request.files.get("logfile")
    if file is None or file.filename == "":
        return upload_page("Please choose a log file first."), 400
    text = file.read().decode("utf-8", errors="replace")
    return show_dashboard(text, file.filename)


@app.route("/sample")
def sample():
    if not SAMPLE.exists():                    # create the sample log if it is missing
        generate_logs.generate(str(SAMPLE))
    return show_dashboard(SAMPLE.read_text(), "api_health.log (sample)")


@app.route("/api/analyze", methods=["POST"])
def api_analyze():
    entries, bad_lines = la.parse_lines(request.get_data(as_text=True).splitlines())
    if not entries:
        return jsonify(error="no valid log lines found"), 400
    return jsonify(summary=la.overall_summary(entries), endpoints=la.analyze(entries),
                   skipped_lines=bad_lines)


@app.errorhandler(413)
def too_large(_error):
    return upload_page("The file is too large. The limit is 5 MB."), 413


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=5000)
```

Output

Running `python log_analyzer.py` prints the following report on the console for the sample log of 1,000 requests:

```
================================================================================
                    API HEALTH LOG ANALYSIS REPORT
================================================================================
Total requests     : 1000
Success rate       : 91.7%
Error rate         : 8.3%
Avg response time  : 619.1 ms (median 301.0, p95 2398)
Malformed lines    : 0

Status code distribution:
  200    531  ####################################################
  201    177  #################
  204    209  ####################
  400      6
  401     12  #
  403     11  #
  404      9
  500     10  #
  502     12  #
  503     11  #
  504     12  #

Per-endpoint health:
  Endpoint          Reqs    Err%   Avg ms   P95 ms  Status
  ------------------------------------------------------------
  /api/payments      176    25.0   1983.2     4644  CRITICAL
  /api/orders        164   13.41    668.8     1478  DEGRADED
  /api/login         157     5.1    178.7      249  DEGRADED
  /api/products      145    2.76    270.8      389  HEALTHY
  /api/search        183    1.64    380.6      587  HEALTHY
  /api/users         175    1.14    133.4      193  HEALTHY

Top 3 most error-prone endpoints:
  1. /api/payments  (25.0% errors)
  2. /api/orders  (13.41% errors)
  3. /api/login  (5.1% errors)

Slow requests (> 1000 ms): 151

Errors by hour:
  08:00  17
  09:00  17
  10:00  18
  11:00  24
  12:00  7

ALERTS:
  [CRITICAL] /api/payments - 25.0% errors, avg 1983.2 ms
  [DEGRADED] /api/orders - 13.41% errors, avg 668.8 ms
  [DEGRADED] /api/login - 5.1% errors, avg 178.7 ms
================================================================================
Saved: report.json, report.csv
```

Website (`python app.py`, then open http://127.0.0.1:5000)

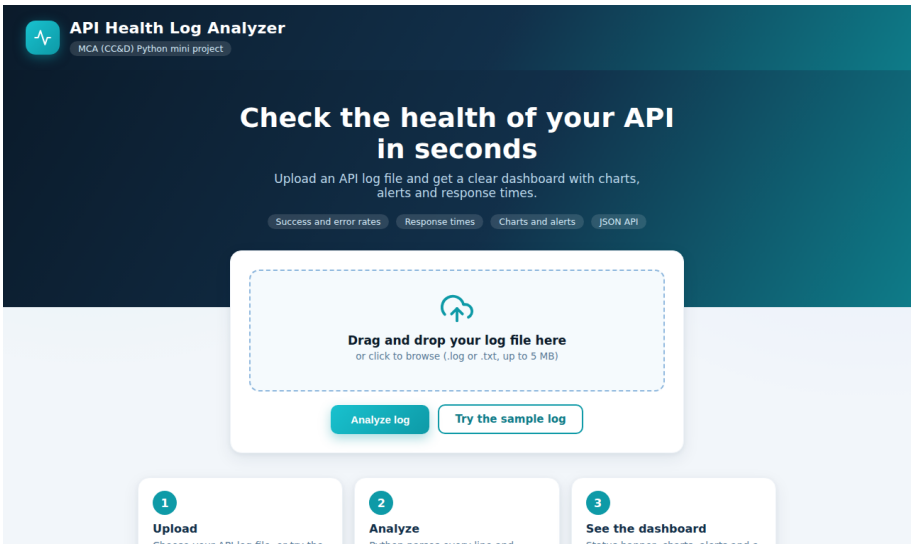


Fig. 1: Upload page of the website

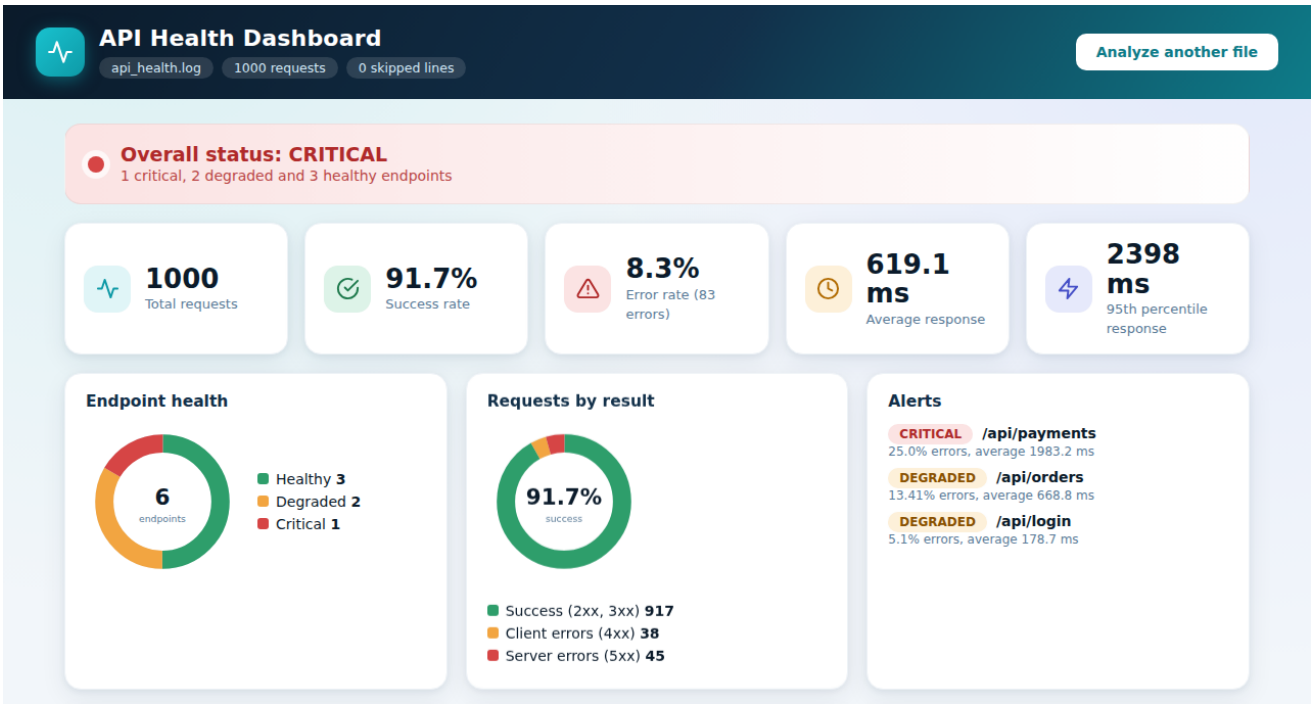


Fig. 2: Dashboard after uploading a log file: status banner, KPI cards, donut charts and alerts

If no file is chosen, or the file has no valid log lines, the website shows a clear error message on the upload page instead of crashing.

Website dashboard (continued)

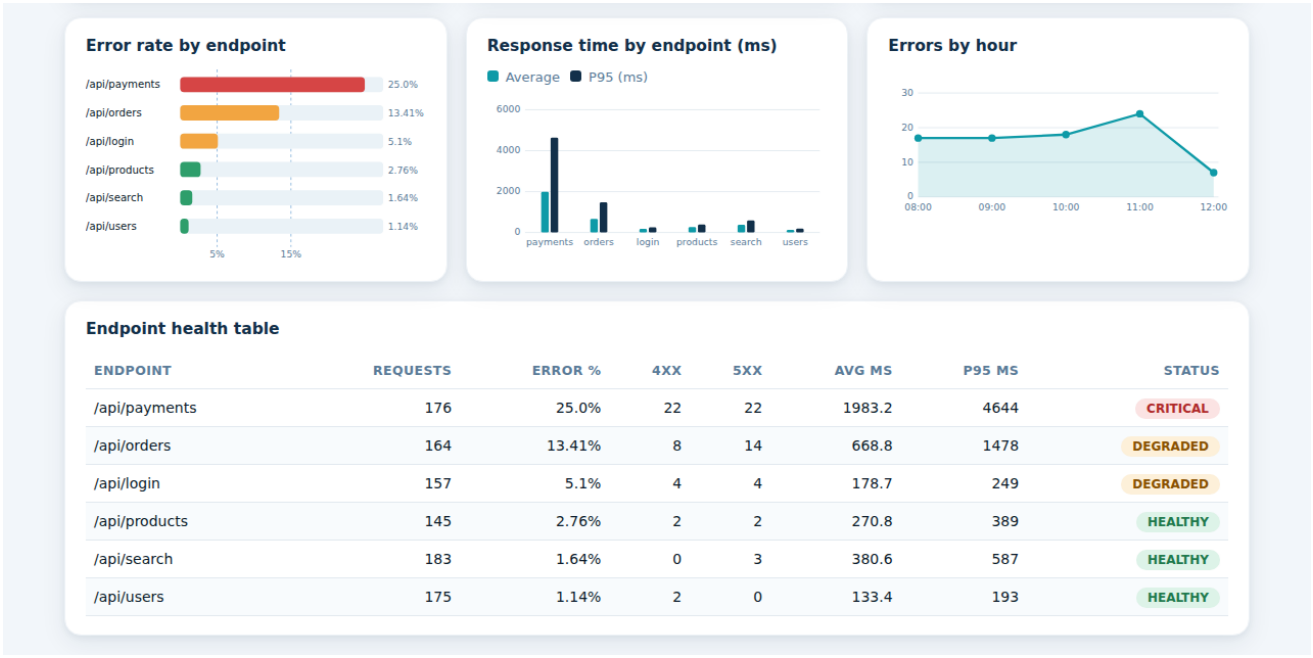


Fig. 3: Error rate, response time and errors-by-hour charts, and the endpoint health table

- **Overall status banner:** the worst status among all endpoints.
- **KPI cards:** total requests, success rate, error rate, average and 95th percentile response time.
- **Donut charts:** endpoints by health status, and requests by result (success, 4xx, 5xx).
- **Bar, column and line charts:** error rate per endpoint with the 5% and 15% limits, average and P95 response time, and errors in each hour.
- **Endpoint table:** requests, errors, response times and a colored status badge.

8. Functionalities

Log Parsing

- Reads the log line by line and extracts the method, endpoint, status code and response time.
- Skips malformed lines and reports how many were found, so one bad line never stops the program.

Metrics Calculation

- Calculates total requests, success rate, error rate and the status-code distribution.
- Calculates average, median, minimum, maximum and 95th percentile response times.
- Counts 4xx and 5xx errors separately for every endpoint and shows errors by hour.

Health Classification and Alerts

- Labels each endpoint HEALTHY, DEGRADED or CRITICAL using adjustable rules.
- Lists the most error-prone endpoints and prints an alert for every unhealthy one.

Report Export

- Saves the full result as `report.json` and `report.csv` so it can be reused.

Website and Dashboard

- Upload a log file in the browser (drag and drop or click) and see its health dashboard immediately.
- The dashboard shows an overall status banner, KPI cards, alerts, donut, bar, column and line charts and an endpoint table.
- All charts are drawn as SVG by Python, so no chart library or internet connection is needed.
- Try the built-in sample log with one click.
- `/api/analyze` returns the report as JSON for other programs.
- Handles missing files, invalid content, files above 5 MB and HTML in log text safely.

Command-Line Options

- `--file` chooses the log file, `--slow` sets the slow-request limit in milliseconds and `--top` sets how many error-prone endpoints are listed.

9. Conclusion

The API Health Log Analyzer successfully turns a large raw log file into a short and readable report. It finds failing and slow endpoints automatically, labels them with a clear status and presents the result as text, files and a dashboard with charts in a working website. The analysis uses core Python and applies first-semester topics such as file handling, functions, dictionaries, regular expressions and exception handling, while Flask turns it into an easy-to-use web tool.

10. Future Work

- **Live Data:** Read real API responses using the requests library instead of a log file.
- **Email Alerts:** Send a notification when an endpoint becomes CRITICAL.
- **Database Storage:** Store logs in a SQL database and analyze them with queries.
- **Deployment:** Host the Flask website online so that a team can use it.
- **User Accounts:** Add login and keep a history of uploaded logs and trends.

11. Learning Outcomes:

- Proficiency in file handling and processing large text data line by line with Python.
- Skill in writing regular expressions to extract structured data from text.
- Use of dictionaries, defaultdict and Counter to group and count data efficiently.
- Ability to calculate and interpret statistics such as mean, median and percentiles.
- Experience in exception handling, command-line programs with argparse and exporting JSON and CSV.
- Experience in building a Flask website with routes, file upload and safe handling of user input.
- Experience in drawing charts as SVG and generating an HTML dashboard from Python.

12. GitHub link:

https://github.com/[your-username]/[repository-name]

13. Bibliography:

- Python Software Foundation. (n.d.). *Python 3 documentation*. Retrieved from https://docs.python.org/3/
- Python Software Foundation. (n.d.). *re - Regular expression operations*. Retrieved from https://docs.python.org/3/library/re.html
- Python Software Foundation. (n.d.). *statistics - Mathematical statistics functions*. Retrieved from https://docs.python.org/3/library/statistics.html
- Pallets. (n.d.). *Flask documentation*. Retrieved from https://flask.palletsprojects.com/
- MDN Web Docs. (n.d.). *SVG: Scalable Vector Graphics*. Retrieved from https://developer.mozilla.org/en-US/docs/Web/SVG